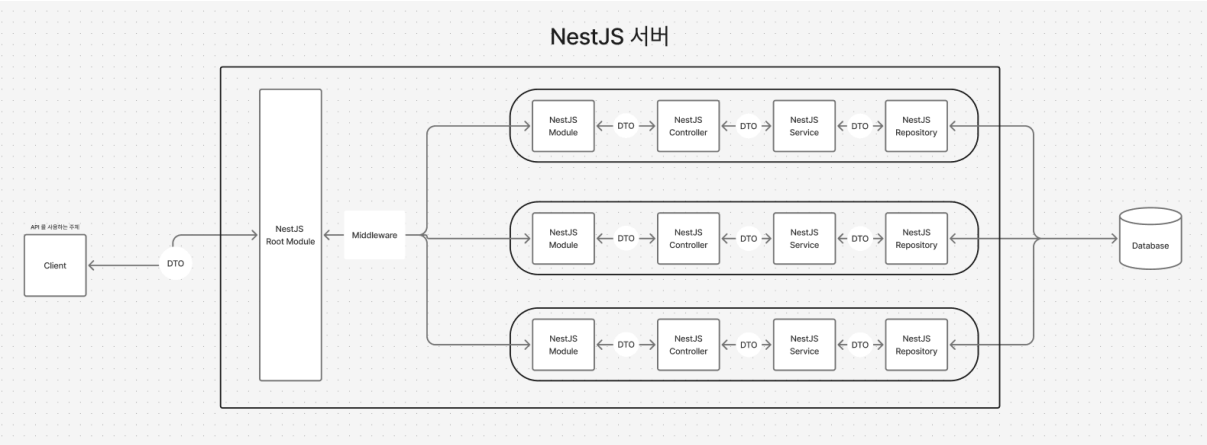
# Middleware

## ✅ Middleware란?

- **RootModule → 기능 Module** 사이에 위치




- 요청(Request)을 먼저 받아 **가공/검사/기록 후** 다음 단계로 전달

> 📌 Controller보다 먼저 실행되는 전처리 레이어

---

## 💡 언제 사용하나요?

- 요청 로그 남기기
- 헤더 검사 / IP 차단
- 공통 데이터 추가

  → 예: `req.requestTime = new Date()`

> 가벼운 전역 처리 작업에 적합

---

## 🛠️ 기본 코드 구조

```
@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: () => void) {
```

```
    console.log(`[${req.method}] ${req.url}`);
    next(); // 반드시 호출!
  }
}
```

```
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer.apply(LoggerMiddleware).forRoutes('');
  }
}
```

> forRoutes('') → 전 요청에 적용

---

## ⚠️ 사용 시 주의할 점

- `next()` 호출 필수
- Middleware는 Express 기반 → express의 `Request`, `Response` 타입 사용
- (함수형 미들웨어의 경우) **DI(의존성 주입)** 제한 있음

---

## 🧩 Middleware vs Guard vs Interceptor

| 항목 | Middleware | Guard | Interceptor |
|---|---|---|---|
| 실행 위치 | 가장 앞단 | 라우팅 직전 | 컨트롤러 전/후 |
| 주요 목적 | 요청 전처리 | 인증/인가 | 응답 조작, 로깅 |
| DI 사용 가능 여부 | 제한적 | 가능 | 가능 |

> ✅ 전처리는 Middleware,
>
> ✅ 인증은 Guard,
>
> ✅ 응답 조작은 Interceptor

---

## 📌 정리

- Middleware는 NestJS 구조에서 **가장 먼저 실행되는 필터**

- 실무에서 **공통 전처리 작업**에 자주 사용
- **너무 많은 역할을 부여하기보단** 가볍게 쓰는 것이 핵심